

15/08

UBA XXI

Cátedra Emanuel Camejo
Pensamiento Computacional (40)

1

Conceptos básicos en programación y pensamiento computacional

Lenguaje técnico: En el proceso de análisis, negociación, definiciones es importante q' los $\neq$ roles de un proyecto compartan el lenguaje técnica, agiliza el dialogo y mejora la comprensión. Evita malos entendidos y frustraciones

"Aprender a programar o pensar como un programador no sólo implica conocer ciertos modelos, herramientas y técnicas, tmb es ! aprender a hablar c/ellos."

La computadora

¿Que es? Todos los programas se corren en una

Es un dispositivo físico de procesamiento de datos c/ propósito general.

La idea de una ctora. es que es capaz de procesar datos y obtener nueva info e rtados ¿Cuáles? Los que se le piden ¿Quien se lo pide? El/los programas que se ejecutan. Los programas q' se deben correr son ilimitados a lo que la tecnología permite

En el dispositivo, puedo instalar una app creada por mí o descargada de internet entonces es una computadora

Software y Hardware

Las computadoras son sistemas que integran 2 aspectos o planos contrapuestos, ambos determinantes en su comportamiento. S y H

dice q' hacer $\leftarrow$ S: Conjunto de herramientas abstractas (programas), componente lógico de un modelo computacional.

lo hace $\rightarrow$ H: Componente físico

Hoy en día la tecnología dom H dominante es la electrónica

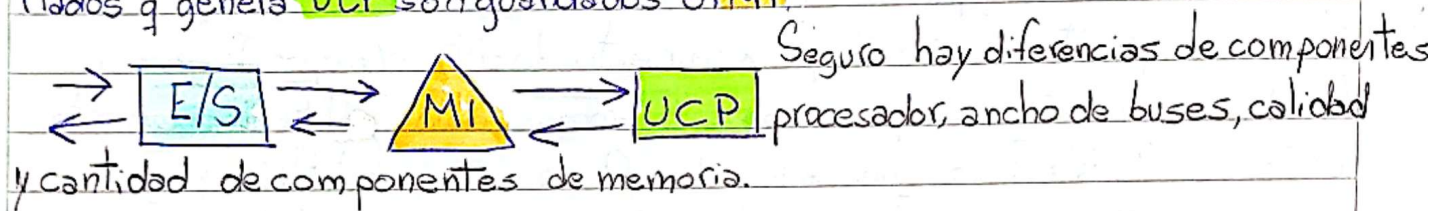
Si mundo de las ideas.
Casi todas las innovaciones en programación, ya fueron pensadas décadas atrás
P/no llegaban al plano H, no lo permitía
pq' el avance del h, permitía

15/08-

• [1] modelo de Von Neuman y la Memoria Interna (MI)

Arquitectura de una computadora: Es el modelo q' su diseño sigue c/ respecto a sus partes o subsistemas y la forma en la que se interrelacionan. La inmensa mayoría son Von Neumann; tiene 3 partes

- ① Info; necesariamente ingresa a través del subsistema de Entrada-Salida (E/S) E: Mouse o teclado, S: pantalla
- ② Todo dato que se necesite p/ el procesamiento debe alojarse en (MI), incluyendo los programas y los datos temporales que se generan
- ③ La Unidad Central de Procesamiento (UCP o CPU) es quien realiza todas las transformaciones de datos, el proceso en sí. Los ritados se envían a los usuarios a través del módulo E/S, p/ este los obtiene de la MI, por eso, los ritados q' genera UCP son guardados en MI.



Al usar esta arquitectura, una instrucción debe estar cargada en MI para ser ejecutada. Como no es cómodo escribir uno por uno los comandos o sentencias antes de ser ejecutados, en la programación moderna se trabaja c/ el concepto de Programa almacenado.

"El Modelo de funcionamiento de Programa Almacenado implica, que para poder ser ejecutado un programa, debe ser cargado previamente en MI."

• El Sistema Operativo

Programa encargado de administrar el equipo y sus recursos. Que constantemente son disputados entre las solicitudes de los procesos q' se ejecutan al mismo tiempo. Las compus. modernas delegan esta gestión en el S.O.

↓ Pueden funcionar sin S.O., p/ ninguna compu. moderna lo intenta, pq no sale bien.

L-Language

C-Computadora A-Algoritmo Pr-Programación D-Dato

UBA XXI

Pensamiento Computacional (90)

2

• Programa "Un P de C es un A escrito en un Language de Pr."

$P = A + D$ | Para escribir un programa, es necesario escribir un A y diseñar una correcta org. de sus datos.

Un buen programador no sólo mira el algoritmo, también hay q' ser inteligente en la selección del modelo y herramientas p/organizar los datos.

• Dato | Dato $\neq$ información $\rightarrow$ asignarle sficado al dato.

"En TI, un D es una representación simbólica ya sea numérica o alfabética cuyo valor está listo p/ser procesado por una C y mostrarlo al usuario a modo de inf."

• Algoritmo - Serie finita de pasos precisos p/alcanzar un objetivo
conjunto ordenado

¿De qué manera debe ser escrito el algoritmo p/que lo comprenda la C? Una C entiende 2 eventos:

- Ausencia & Presencia de tensión ^{estados} disjuntos y complementarios

Bit: Cero & uno. Dígito binario. Byte- 8bits, Kbyte 1024 bytes, Megabyte 1024 Kbytes

1. Analisis del problema
2. Primer esbozo de la sol.
3. División del prob. partes
4. Ensamble de las parte

La C almacena y procesa info. en forma de unos y ceros, que manejan $\neq$ unidades al agruparse en ciertas cantidades. Cualquier algoritmo o instrucción que se le da a la computadora debe estar codificado de manera tal, ^{q'se} pueda representar en ceros y unos.

- P/ lograr que un algoritmo escrito a humanas sea entendido por la C, hay que usar un L

• Language de Programación | Un language es un protocolo de comunicación

Protocolo: Conjunto de normas consensuadas.

Si un language es comprendido por el humano como por la máquina = ^{comunicación} efectiva

Language Assembler, primitivo

→ Lenguajes formales: Tienen un conjunto acotado y definido de elementos, oraciones, reglas de sintaxis. Ej. Lenguas vivas, leng. matemático, partituras musicales. Ej. no formales: emojis, múltiples sficados.

La C es completamente literal al momento de leer las instrucciones.
Hoy día los lenguajes ^{depr} se aproximan a los coloquiales. De todas formas, el
leng. en el q' yo programo debe ser **TRADUCIDO** a **ASSEMBLER**.

• TRADUCTORES

Escribimos programas en lenguaje de Alto Nivel, ^{cercano a código máquina} y de Bajo Nivel y los
que deben ser traducidos a BN. por otro programa traductor. **Intérpretes & Compiladores**
Compiladores: Leen y después traducen completamente el archivo, y sólo entonces
puede ser ejecutado.

Intérprete: Traduce sentencia a sentencia a medida que se solicita ^{parten los} en ejecución.

• Los programas escritos en AN, son los **P fuente**. P' desde allí p' generar una **V ejecutable**.
Un **P fuente** es un archivo de instrucciones.

Existe un abismo entre los programas diseñados por el programador y la
realidad de quien tiene que ejecutarlos (la máquina), es cada vez mayor.

Admins. de dispositivos, transmisiones de info., admins de recursos propios,
orgs. de tareas, S.O., graficadores, traductores, compresores, ambientes de
diseño, van resolviendo problemas p' que las apps de N.A. pueden abordar
la resolución de problemas complejos, despreocupándose de detalles.

• **Python** - Lenguaje AN, multi-paradigma, multi-plataforma

M-Pa: pueden usar $\neq$ paradigmas, enfoques en un mismo o $\neq$ proyectos

M-Pl: Un programa en py puede ejecutarse en $\neq$ SO.

Sentencias básicas y datos simples

"En este curso la comunicación de nuestros programas con el mundo exterior se realizará casi exclusivamente con el usuario; tanto sea p/ obtener datos de entrada, como para comunicar ritados parciales o finales"

1.2 Funciones

- Permitir que el usuario ingrese datos `input()`
- Mostrar info al " `print()`

1.3 Variables y constantes

No se puede usar el mismo nombre p/ 2 variables. Se pisa el dato anterior

1.4 Buenas Prácticas

Python lee, línea a línea el código. Lo q' puede ejecutar, lo ejecuta. Las fn las guarda en memoria p/ usar luego.

2. Tipos de datos

2.1 Datos Simples

Tipo de dato	Ej.
int	9, -5
float	0,01 88.197
complex	10,9j // la j del par indica la parte imaginaria
bool	true // false
str	'~' " 'hola'

* es simple, pero tmb una colección de datos

2.

2.2 Operadores

			Abreviada			Abreviada	
//	Div. Entera	a//12	/=	Div. Entera	a/=3 { a=a/3		
+	Suma	10+cant	%	Resto	num-par%2	//=	Div. Entera a//=3 { a=a//3
-	Resta	saldo-pago	+=	Suma, agrega	a=a+3 { a+=3	%=	Resto a%=3 { a%=a%3
*	Producto	a*20	-=	" , agrega	a-=3 { a=a-3		
/	Div. precisión decimal	a/3.5	*=	Prod, "	a*=3 { a*=a*3		

Para alterar cualquier precedencia, se usan $()$, como en matemática.

1. parentesis()

2. *, /, //, %

3. +, -

** > * > +

Operadores de edición de texto

+	Concatenación	'hola' + 'pepe' → 'holapepe'
*	Repetición de texto	'ja' * 3 → 'jajaja'
[k] o [-k]	Selección de carácter	a = 'hola' a[0] → 'h' a[2-1] → 'o'
[i:j:p]	" de una posc. o porción	a[0:2] → 'ho'
+=	Concatenación de texto	a += 'y chau' → 'holay chau'
*=	Rep. abreviado	a * 2 → 'holahola'

4. ()

5. []

6. +, *

2.2.1 Manipulando strings Empiezan desde cero

Rangos: Un rango tiene 3 partes

[start: end: step] → siquiero q' se saltee posiciones
dnd comienza hasta dnd, sin incluir

2.3 Input y casteos / Parsear

int() (str()) float()

3 Funciones → Python tiene fns nativas, p/ podemos crear las nuestras

nombre = " ~ "

```
def obtener_saludo(nombre):  
    return "Hola, " + nombre
```

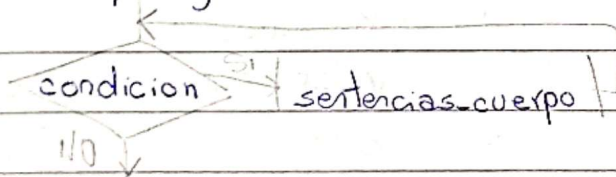
} Esta es la fn

saludo = obtener_saludo(nombre) → Guardo la ejecución en una variable
print(saludo) → Si no la muestro, no veo nada.

Funciones predefinidas de python

print()	
input()	
abs()	Valor absoluto de un nro
round()	
int()	
float	
str()	
bool()	
len()	
max()	" máximo "
min()	Valor mínimo entre elementos o secuencias
pow()	Potencia de un nro
range()	Genera una secuencia de nros
type()	Devuelve el tipo de un obj.
round()	Redondea a una cant. específica de decimales
isinstance()	Verifica si un obj. es instancia de otro
replace()	Reemplaza todas las apariciones de un subtexto

Ciclos y rangos



2 While

```

total_multiplos = 0;
condicion_de_corte = 0;
while condicion_de_corte < 5:
    num = int(input('Ingresa 5 nros enteros'))
    if num % 3 == 0:
        total_multiplos++
    condicion_de_corte++
print("total multiplos")
  
```

Tmb se puede hacer un bucle dnd hasta que no ingreses la palabra secreta, no se termina el bucle.

→ Hay muchas formas de hacer la misma condición.

2.2 Break y Continue → Controlan el flujo del programa

2.2.1 Break → P/salir inmediatamente de un bucle antes q' complete su iteración

nro = 10

while nro < 30:

if nro < 30:

print("El 1er nro múltiplo de 3 es, nro")

~~break~~ → Tenía esta línea mal indentada, y no funcionaba. Se salía a la 1ra

nro += 1 Sin esta línea, se ejecuta bien

2.2.2 Continue → P/saltar una iteración del bucle y continuar a la siguiente. Se salta log este debajo suyo

CONTINUE

nro = 1

veces = 0

while nro <= 10:

if nro % 2 == 0:

Si es PAR
ejecuta esto
print("if, ANTES", nro)
nro += 1
continue

[continue] print("if, DSPS", nro)

Si no es PAR

print(nro)
nro += 1
veces += 1

print('dps del bucle', nro)

print(" " " " veces', veces)

Iteración	nro al entrar	¿Es par?	Acciones / Imprime	nro al salir	veces
1	1	X	1, suma 1	2	1
2	2	✓	"if, ANTES" 2, suma 1 if, DSPS 3, suma 1	4	2
3	4	✓	if ANTES 4, suma 1 if DSPS 5, suma 1	6	3
4	6	✓	if ANTES 6, suma 1 if DSPS 7, suma 1	8	4
5	8	✓	if ANTES 8, suma 1 if DSPS 9, suma 1	10	5
6	10	✓	if ANTES 10, suma 1 if DSPS 11, suma 1	12	6

CON CONTINUE

Iteración	nro al entrar	¿Es par?	Acciones / Imprime	nro al salir	veces
1	1	X	1, suma 1	2	
2	2	✓	if ANTES 2, suma 1	3	1.
3	3	X	3, suma 1	4	2
4	4	✓	if ANTES 4, suma 1	5	2
5	5	X	5, suma 1	6	3.
6	6	✓	if ANTES 6, suma 1	7	3
7	7	X	7, suma 1	8	4
8	8	✓	if ANTES 8, suma 1	9	4
9	9	X	9, suma 1	10	5
10	10	✓	if ANTES 10, suma 1	11	5

sólo entra, pasa por veces cuando NO es par

3. For El ciclo finaliza como no hay + valores p/ iterar
for item in items: iterable deben existir valores dentro.

3.2 Rangos → Es un constructor o estructura de control q' permite generar una sucesión de nros enteros, con un cierto paso o salto regular

range(valor_inicial, valor_final, paso) debe ser un entero, sino se coloca es 1.
si no se coloca, se asume 0 no se incluye Si se coloca, debe colocarse valor inicial si se incluye P/ rango decreciente, paso negativo

range(1, 10) # 1, 2, 3, 4, 5, 6, 7, 8, 9

range(6) # 0, 1, 2, 3, 4, 5 → Hay 6 elementos

hay 8 elementos

range(0, 8, 2) # 0, 2, 4, 6 → Es 0-7, y no pasos Si no estuviera el 'paso' iría de 0-7.

range(15, 10, -5) # De 15 a 10, sin incluirlo. En paso de -5. Como al 10 no llega, solo muestra 15

range(15, 10, -1) # De 15 a 10, En paso de -1 15, 14, 13, 12, 11.

4. Anidados

Nros primos

Ej. de bucles anidados: Calcular en base al nro q' ingresa el usuario, sus divisores.

• Mientras el nro ingresado sea != de cero p'q' cero no tiene divisores

• Todo nro N es divisible por 1 y por sí mismo. Así que no se cuentan

• El máximo divisor de un nro es: el nro dividido 2.

num_user = int(input("Ingresa un nro mayor a cero"))

NO

while (num_user > 0): Ej: Si yo ingreso 10. $10 // 2 = 5$ El rango de 2, 5

cantidad_divisores = 0;

mitad_de_num_user = num_user // 2;

rango_de_nros_a_evaluar = range(2, mitad_de_num_user + 1)

for num in rango_de_nros_a_evaluar:

if num_user % num == 0:

cantidad_divisores += 1

print("cantidad_divisores:", cantidad_divisores)

num_user = int(input("Ingresa un nuevo nro, o uno menor a cero p/ salir"))

Lo que yo quiero saber es si el nro es divisible por el nro q' ingreso el usuario

1 Condicionales | Las preguntas q' un programa necesita formular deben estar escritas como lógicas. Son condiciones

1.1.2 Operadores Lógicos

Negación ~	"No", si niego algo V, lo transforma a falso. Y viceversa	not
Conjunción ^	"Y". Sólo da V si todas las condiciones son V.	and
Disyunción v	"O". Da V si alguna de las expresiones que evalúa es V	or

AND			OR			NOT	
A	B	Rtado	A	B	Rtado	A	R
V	V	V	V	V	V	V	F
V	F	F	V	F	V	F	V
F	V	F	F	V	V		
F	F	F	F	F	F		

Si no hay alteraciones de precedencias, se priorizan los AND sobre los OR.

- $V \text{ or } F \text{ and } F$ equivalente $V \text{ or } (F \text{ and } F) = V \text{ or } F = V \rightarrow *$ En expresiones combinadas AND le gana a OR.
- $V \text{ or } F \text{ or } F$ equivalente $(V \text{ or } F) \text{ or } F = V \text{ or } F = V$
- $V \text{ and } V \text{ and } F$ equivalente $(V \text{ and } V) \text{ and } F = V \text{ and } F = F$

- $(V \text{ or } F) \text{ and } F = V \text{ and } F = F \rightarrow *$ Cambia
- $V \text{ or } (F \text{ or } F) = V \text{ or } F = V$
- $V \text{ and } (V \text{ and } F) = V \text{ and } F = F$

Tanto OR como AND, son asociativos.

1.2 If

if condicion_1:

elif condicion_2:

else:

1.3.1 Operadores de comparación de desigualdad

==	igual
!=	distinto
<	menor
<=	menor igual
>	mayor
>=	mayor igual

Operadores de pertenencia

in	pertenece	"vazon" in "corazon" # True	"tazon" in "corazon" # False
not in	NO pertenece	" not in " # False	" not in " # True

Secuencias, Tuplas y Listas

1 Secuencias - Serie de elementos que se suceden y guardan relación entre sí

- Listas - Tuplas - String - Rangos

- Tuplas: Inmutable, ≠ tipos de datos

(8,) → Tupla (8) → elemento (a,b), b = (a,b), b

('~',) → tupla ('~') → elemento

Ej: a = (8, 2, 5) a[0] = 1
error

¿Cómo modificar una tupla? Las tuplas se crean d/valor.

a = (1, 2) → a = a * 3 // (1, 2, 1, 2, 1, 2)

- Listas: Mutables

a = [1, 2, 3] → 009
b = [1, 2, 3] → 008
a = [1, 2, 3] → 009
b = a → al cambiar b, cambio "a"

✓ b = a.copy() → 008

✓ lista = [1, 2, "~"]

lista[2] = "~" → En tuplas no se puede

Operadores de Secuencias → P/todos

<code>x in s</code>	Devuelve <u>True</u> si <code>x</code> pertenece a <code>s</code> , sino <u>False</u>
<code>s + t</code>	Concatena
<code>s * n</code>	Concatena a <code>s</code> , <code>n</code> veces
<code>s[i]</code>	pos <code>i</code> de secuencia <code>s</code> → elem. en
<code>s[-k]</code>	<code>k</code> posiciones antes del final → elem. en
<code>s[i:j]</code>	porción entre <code>i</code> y <code>j-1</code>
<code>s[i:j:k]</code>	porción entre <code>i</code> y <code>j-1</code> , de a <code>k</code> elementos [desde: hasta: de a paso]
<code>>, <, >=, <=, !=</code>	comparación
<code>len(s)</code>	longitud
<code>min(s)</code>	mínimo elemento
<code>max(s)</code>	máximo "
<code>sorted(s)</code>	devuelve <code>s</code> ordenada en forma de lista
<code>reversed(s)</code>	" <code>s</code> invertida
<code>s.count(x)</code>	" nro total de ocurrencias de <code>x</code> en <code>s</code>
<code>s.index(x, i, j)</code>	<code>s.index(20, 2, 4)</code> <code>s = [10, 20, 30, 20]</code>
$\overbrace{j-1}^{j-1}$	busca desde hasta SIN INCLUIRLO

Metodos para Lista

myList

<code>append(m)</code>	<code>.append(s)</code>	agrega al final
<code>insert(pos, val)</code>	<code>.insert(2, 10)</code>	En la pos, inserta. Mueve todo el array b
<code>remove(val)</code>	<code>.remove(7)</code>	Quita este elemento de la lista
<code>pop([índice])</code>	<code>val = .pop(3)</code>	Quita ese índice
<code>extend(otra lista)</code>	<code>.extend([8, 9])</code>	→ Agrega los elementos de otra lista, al final de esta
<code>sort()</code>	<code>.sort()</code>	→ Por defecto de menor a mayor
<code>sorted()</code>	<code>a = sorted(a)</code>	ambos reciben 2 parámetros <code>key=fn</code> y <code>reverse=True/False</code>
<code>reverse</code>	<code>.reverse()</code>	<code>a[::-1]</code> → No retorna la lista modificada → Retorna None
<code>count(val)</code>	<code>.count(s)</code>	cuenta las apariciones de <code>s</code> en <code>D</code>
<code>index(val)</code>	<code>ind = .index(8)</code>	índice de la 1ª aparición
<code>len()</code>	<code>len()</code>	longitud (elementos totales)
<code>clear()</code>	<code>.clear()</code>	
<code>copy()</code>	<code>.copy</code>	
<code>max()</code>	<code>max()</code>	
<code>min()</code>	<code>min()</code>	
<code>sum</code>	<code>sum()</code>	

LISTAS $\neq$ tipos $\rightarrow$ mutables
 TUPLAS - STRING $\rightarrow$ inmutables

```
nombres = []
nom = input('ingrese * p/salir')
while nom != '*':
    nombres.append(nom)
    nom = input(' ')
for n in nombres:
    print('---> ', n)
```

si no, hasta el final

```
from random import randint
a = []
for i in range(20):
    a.append(randint(1, 35))
for i in a[10:]:
    print(n, end=' ')
for i in a[:10]:
    print(n, end=' ')
print()
```

entre

desde 0 cero, hasta SIN incluirlo

reformato la salida

1.6 Métodos de la secuencia str (string)

"str" "python"

capitalize	.capitalize()
center(lancha,relleno)	.center(10, '*')
count(buscar)	.count('a')
find('substr', desde, hasta)	.find('th', 1, 4)
rfind('~')	.rfind('~')
format(args, args...)	"{} {}".format(3, 10)
upper()	upper
lower()	lower
strip	.strip
replace("es", "por", "esto")	.replace("y", "i")
split(" ") $\rightarrow$ separador	.split(",")
join([" "]) $\rightarrow$ Unido por (n)	"manzana,banana,uva" $\rightarrow$ split(",") $\rightarrow$ ["manzana"]
isdigit() true si todos los caracteres son dig	123.isdigit()
isalpha() true si todos los caracteres son alfa numer	"mana,banana"
startswith('~')	
index	
ljust/rjust	
str.maketrans	
translate	

va a buscar esto en el string, y va a crear pos de arrays

Una, lo q le paso entre (), separado por " "

sort y sorted

Diccionarios

a = str.maketrans("aeiou", "12345")

Asocia c/ vocal a un nro:

97	101	105	111	117
a	e	i	o	u
1	2	3	4	5
49	50	51	52	53

{97:49, 101:50, 105:51, 111:52, 117:53}

"hello".translate(a) → h2ll4 solo cambia los caracteres del diccionario q conoce

Las tuplas NO son mutables c/ las listas.

Hay métodos list(~) y tuple(~) que convierten lo q pasas en los
(~) en lista o tupla.

Repasar: filter

Questionario Sesión 4

String → Exclusivo de caracteres

Tuplas → Cualquier tipo de dato } No retorna nada

→ Si la lista ya está ordenada, devuelve None

} Solo acepta listas

! sort() → modifica la lista original p/ NO devuelve nada } Alfabética y ascendente mente

map y filter toman una fn y una secuencia c/ param ✓

PERO

map → Transforma c/ elemento de la secuencia

filter → selecciona los elementos en base a la fn booleana administrada

! sorted → Devuelve una nueva lista } Asc
→ Acepta cualq. elemento iterable

list.sort(key = myfn, reverse = True)

cars = ['Ford', 'BMW', 'Volvo']

cars.sort() // BMW, Ford, Volvo

cars.sort(reverse = True) // Volvo, Ford, BMW

def miFn(e): es el parámetro de la fn

return len(e) → Según el largo de la palabra

cars.sort(key = miFn) // BMW, Ford, Volvo

def porAño():

return e['year']

cars.sort(key = porAño)

cars = [{'w': 'w', 'year': 2000},
{ 'w': 'w', 'year': 1999}]

✓ Sesión 08

Archivos Se conocen cómo

8

Estructuras de datos persistentes → Significa que perdura más allá de la vida de quien las genera. Un archivo debe guardarse en un dispositivo que administre datos no volátiles (Que no desaparezcan cuando se apaga el equipo)

Si genero un grupo de datos, por teclado o cómo estado de transformaciones, necesito guardarlo en una Estructura capaz de sobrevivir a la ejecución de corriente (proceso ^{activo} ~~arch~~). De esta manera, se puede seguir trabajando c/ellos posteriormente

Todos los leng. de pr. admiten archivos

- Estamos haciendo programas que corren en una Arquitectura de Von Neumann. Por lo tanto, p/q' los datos puedan ser procesados, primero deben ser alojados en la **Memoria Interna**.
- Y p' que se puedan sacar datos y estados (de la computadora hacia dispositivos externos (discos, redes), donde estará alojado el archivo, también debe pasar **MI**

~ Así que no hay posibilidad de trabajar/ los datos q' están en un archivo si no los copio a **MI**. Si lo modifico y deseo guardar las modificaciones, debo bajar la V. de **MI** al archivo

Normalmente los archivos contienen volúmenes de datos muy grandes, por eso copiar un archivo en **MI** puede ser muy costoso o imposible. Una solución es copiar el archivo por partes, a medida que se necesita, se sube a **MI** y a medida que se modifica, guardo una copia en el archivo

El S.O. reserva una zona de memoria y establece un canal de comunicación (entre el archivo y el programa. Se denominan buffers

→ Implementa un sist. de tráfico de intercambio, lectura y escritura en bloque. O sea, si el programa necesita un dato particular, no trae solo ese dato, sino todo un bloque de info dentro del cual, se incluye el dato.

Abrir un archivo, se traduce al S.O. cómo pedirle que defina un buffer. Este va a tener el nombre, donde se ubica, qué hará c/él y más cosas y datos.
 Se le asigna un nombre interno (alias) a c/archivo p/ referenciarlo c/ ese nombre dentro del programa.

alias = open(nombre_file, modo) ^{→ completa}
 alias.close()

1.2 Modos de apertura				de Lectura	de Escritura
Modo	Lee	Escribe	Existencia de Archivo	Posc. inicial	Posc. inicial
r	✓		Si el archivo no existe, da error	Inicio	-
w		✓	Si el archivo " ", lo crea	-	Inicio
a		✓	Si existe, le borra td el contenido	-	-
a		✓	Si el archivo no existe, lo crea	-	Final
rt	✓	✓	" " " ", da error	Inicio	Depende
wt	✓	✓	" " " ", lo crea	Inicio	Depende
		✓	Si existe le borra td el cont.		

Modo r → por defecto

read → Lee todo el archivo y lo devuelve un string.

readlines → Lee todas las líneas y las devuelve c/ elementos de una lista. Cada string tiene el carácter /n de forma explícita

readline → lee de a una línea c/ vez q' se llama a la fn. Cada línea contiene el carácter /n especial, de forma implícita.

Modo w → no lee, w+ sí | → Si el archivo no existe, lo crea. | → Si existe le borra td el contenido

write recibe un string y lo escribe en el archivo. Si no agrego /n, escribe una palabra detrás de otra.

```

archivo = open("~.txt", "w")
archivo.write('hola')
" .write('chau')
" .close()

```

Resultado: hola chau

`writelines` → puede recibir un string o una lista de strings. De todas formas, si no le agrego el carácter especial `\n`, va a imprimir una palabra detrás de otra.

```
file = open("~.txt", "w")
file.writelines(['hola', 'adios'])
file.writelines('chau')
file.close()
```

Resultado: holaadioschau

! Modo `a` → `append` } Se usan los mismos métodos de escritura que con `w`, la diferencia es que `a`, siempre agrega el contenido al final del archivo.

Problema `c/append` } `p/ escribir en otro renglón:` lo mejor sería que siempre que se escribe, vaya un `\n` al final.

Si mi archivo es así: `append` se posiciona acá `p/ escribir el archivo.`

Modo `r+` → Este modo y el `w+`, son especiales y hay que tener cuidado.

Hay 3 posibles cosas:

Lectura: `r+` } Si sólo leo, `r+` se comporta como `r`. `file = open(" ", "r+")`
`lines = file.readlines()`

Escritura: Empieza a escribir desde el comienzo del archivo. `file.write('Pisa todo')`
 Pisa, carácter por carácter el contenido previo de `c/ línea`. `Texto previo` Pisa todo previo.

Lectura y escritura: Independientemente de qué haga antes, lo que escribo en el archivo va a ir siempre al final. Esto es porque, una vez se lee el archivo, el puntero pasa al final. Es como, "quiero quedarme c/ lo que el archivo ya tiene".

! Si escribo antes y dps leo, igualmente la línea que escribí NO va a estar. Pq los `write` se guardan en el momento en el que se cierra el archivo.

Modo w+ → Igual que con w: Si no existe crea, y si existe lo vacía.

Lectura: Lo 1º q' hace es vaciar el archivo al abrirlo.

```
file = open("archivo.txt", "w+")
line = file.readlines()
file.close()
```

archivo.txt

línea 1

línea 2

Estado:

archivo.txt

[]

Sea cual sea el archivo,
lo deja como []
contenido

Escritura: Vacía por completo el archivo al abrirlo, escribe en un archivo vacío

```
file.write('Nueva línea')
file.close()
```

Estado:

Línea nueva

Lectura y escritura: Como al leer el archivo se vacía, leer antes de escribir c/w+, no devuelve nada.

Si quisiera escribir, y luego leer, lo mismo. El archivo se vacía al abrirlo, el estado es el mismo. Una forma de lograr leer el archivo sobre el q' acabo de escribir, sería posicionarse c/el comando seek(0) en el inicio del archivo, p/ no se ve en la materia.

1.2.7 Otra forma de abrir archivos → Sin usar close "with open as"

with open("archivo.txt", "r") as file:

```
lines = file.readlines()
print(lines)
```

Todo el código dentro del bloque with, va a mantener el archivo abierto, luego se cierra. Como un bucle

1.3 Tipos de archivo | Planos .txt

vaca.txt = open('archivo.txt', 'r') → Lectura = q' (r)

f = vaca.txt.readline() → Lee de a una línea c/ vez q' se llama a la fn
print(f)

f = input('Ingresa un texto c/ vaca:')

while (f.lower()).count('vaca') == 0: → Mientras la palabra ingresada NO contenga 'vaca', que el count de 0

f = input('Ingresa un texto c/ vaca:') → Le pide que ingrese, y no sale hasta q' count = 1, → 1 = 0, False

vaca.txt.write(f + "\n") → Escribe en el archivo, lo ingresado

vaca.txt.close() → Al cerrarse el archivo, se guardan los cambios de write

vaca.txt = open('archivo.txt', 'r') → Si no es específico, es (r) por defecto.

todas = vaca.txt.readlines() → Devuelve un array, donde c/ elemento es una línea del archivo. c/\n explícito

for linea in todas:

print(linea.strip('\n')) → Printea c/ línea sin el carácter explícito \n

1.3.2 Archivos planos - Comma Separated Values CSV

Desde excel se puede guardar una hoja c/ formato CSV.

A punto

```
completo = open('~ / datos Completos.csv') // Crea el archivo datos Completos.csv
```

```
b = [] // Aquí se guarda lo de dat1,2,3
```

```
dat = open('~ / datos1.csv') // Se abre en modo sólo lectura
```

```
a = dat.readlines() // Un array donde c/ línea del archivo es una posición.
```

```
dat.close() // Se cierra
```

len(a) = 6 → posiciones de 0-5 ✓

```
for i in range(len(a)): // Rango del cero a uno menos del largo de a
```

```
    a[i] = a[i].strip('\n') // Quita \n de c/string
```

```
    a[i] = a[i].split(',') // Arma tabla quitando ;
```

```
    a[i][3] = int(a[i][3]) // La columna 3, la convierte en entero
```

```
b = b + a // Suma las líneas de a modificadas
```

```
print(a) // [['Luciana', 'juarez', 'CABA', 95]]
```

// Ordenar y escribir

```
b.sort(reverse=True, key=lambda b: (b[3], b[2])) // Ordena la "tabla" b, q' tiene los datos
```

```
    elemento[3] = str(elemento[3]) // Convierte col 3 en string
```

```
    ele = ','.join(elemento) + '\n' // Une los elementos del, las posiciones, c/ ;
```

```
    completo.write(ele)
```

```
completo.close()
```

datos1.csv

Luciana, juarez, caba, 95

Julian, manzur, pb, 100

Consigna: Ordenar de menor a mayor los puntajes

split → str a []
join → [] → str
posc. del arr.

Ver si hay lambda en parciales

conversión

datos_unificados = open('~ / datos_unificados', 'w')

datos_finales = []

LU, juarez, CABA, 95

LU, manzur, SL, 98

datos1 = open('~ / datos1.csv', 'r')

datos1_array_de_lineas = datos1.readlines()

for linea in datos1_array_de_lineas: → Crudo

linea = linea.strip('\n')

LU, juarez, CABA, 95

linea = linea.split(',') ['LU', 'juarez', 'CABA', '95']

linea[3] = int(linea[3])

datos_finales.append(linea) → limpio

Crea una posición del arreglo por c/coincidencia ('') en el str

def ordenarPorPuntaje(alumno)

return(alumno[3]) → Que ordene por la posición 3

ordenar

datos_finales.sort(key=ordenarPorPuntaje, reverse=True)

print(datos_finales)

for item in datos_finales:

item[3] = str(item[3])

item_unido_a_str_con_n = ','.join(item) + '\n'

print(item_unido_a_str_con_n)

datos_unificados.write(item_unido_a_str_con_n)

datos_unificados.close()

10

Diccionarios - CO7

En las listas se accede por posición. Si preciso un dato específico, y no sé dónde está, 1º debemos averiguar su posición, y luego acceder a la posición.
 — En muchas oportunidades, la situación será acceder por contenido y no por posición.

Un diccionario (tipo dict) es una estructura q' permite mapear una clave a un grupo de datos. "El dato como clave de acceso"

→ Ubica sus elementos en memoria de acuerdo al dato q' obtiene al aplicar una fn de cálculo a su clave. De ahí "Estructura hashable", c/ acceso hash. Nombre q' se le da a los algoritmos de cálculo de direcciones p/ acceso directo a la inform.

$a = \{1:1, 2:4, 3:6\}$ $\text{print}(a[3]) \# 6$

$d = \{345:('clavos', 1), 202:('manguera', 2)\}$ $(d[202]) \# ('manguera', 2)$

- $\text{in}()$ y $\text{len}()$ p/ determinar si una det. clave existe
 → si el dict. está vacío.

El tipo dict es mutable, se pueden agregar o quitar elementos
 $\text{diccionario}[\text{nueva_clave}] = \text{valor}$

- ! Las claves de un diccionario pueden ser números, booleanos, strings, tuplas y cualquier tipo de dato INMUTABLE. No pueden cambiar pq sino el dato de la fn hash aplicada a ellas cambiaría y se debería reubicar la info.

— Admiten heterogeneidad de tipos de claves y de valores

$d = \{1:'yo', 'yo':2, 'ello':(1,2,3)\}$

$\text{print}(d[1], d['ello'][0], d['yo']) \# \text{yo } 1 \text{ } 2$

Metodos p/ diccionarios

`d.clear()` Elimina todos los elementos de `d`.

`d.pop(clave)` Remueve el par clave-valor, y devuelve el valor. Si la clave no está da error

`d.popitem()` Remueve y devuelve cualquier par clave-valor

`d.fromkeys(sequencia)` Crea el diccionario `d`, tomando las claves de una secuencia

`d.items()` Devuelve una lista c/tuplas. C/ tupla tiene elemento 0 clave, elemento 1 valor

`d.keys()` Devuelve una lista c/ las claves de `d`.

`d.update(b)` Agrega los pares clave, valor de `b` a `d`. Si alguna clave existe actualiza su valor

`d.copy()` Devuelve una copia de `d` en otra región de la memoria.

`d.get(clave)` " un valor asociado a una clave

```
def sacaAcent(t):
    con = "á é í ó ú"
    sin = "aeiou"
    traductor = str.maketrans(con, sin)
    return t.translate(traductor)
```

```
txt = input('Ingresá un texto')
txt = sacaAcent(txt.lower())
print(txt.capitalize())
```

%d, %s, %f		
%d	nro entero (int)	'Código: %d' % 10 → 'Código 10'
%s	texto (str)	'Producto: %s' % 'Pan' → 'Producto: Pan'
%0.2f	nro dec (float)	'Precio: %0.2f' % 12,5 → 'Precio: 12,50'
		2decimales
%f		'Precio: %f' % 12,5 → 'Precio: 12,500000'

10/10 -


```
productos = {}
```

```
cgo = int(input('Ingrese código, 0 p/terminar') → C/ esto entra o no al bucle
```

```
while cgo != 0:
```

```
    if cgo not in productos: Descripción de 11: velitas (es el input)
```

```
        desc = input('Descripción de %d: ' % cgo) ↑
```

```
        unidad = input('Unidad de medida %s: ' % desc)
```

```
        precio = float(input('Precio unitario %s: ' % desc))
```

```
        productos[cgo] = (desc, unidad, precio)
```

```
{11: ('velitas', 'unid', 32, 5), 25: ('azucar', 'gr', 0, 65)} C/ esto vuelve o no
```

```
cgo = int(input('Ingrese código, 0 p/terminar') → a entrar al bucle
```

```
for cgo in productos:
```

Desempaquetado
de tuplas

```
    print(cgo, *productos[cgo])
```

```
total = 0
```

```
△ cgo = int(input('¿Que lleva? 0 p/salir: '))
```

```
while cgo not in productos and cgo != 0: # Si el código no existe, vuelve a pedirlo
```

```
while cgo != 0: Si: cgo = 11 | productos[11] = (velitas, unid, 32, 5)
```

```
    cant = float(input('Cantidad de %s: ' % (productos[cgo][0])))
```

```
    total += cant * productos[cgo][2]
```

```
    cgo = int(input('¿Lleva algo más? 0 p/salir'))
```

Mientras el código

ingresado no exista en el dicc cgo = int(input('¿Que más lleva? 0 p/salir')):

productos y no sea cero, sigue pidiendo.

Vuelve a ejecutar este bloque

```
print('Debe abonar: $%.2f' % total)
```



```

productos = {}
cgo = int(input('Ingrese código, 0 p/terminar') → C/ esto entra a no al bucle.
while cgo != 0:
    if cgo not in productos:
        Descripción de 11: velitas (es el input)
        desc = input('Descripción de %d: ' % cgo)
        unidad = input('Unidad de medida %s: ' % desc)
        precio = float(input('Precio unitario %s: ' % desc))
        productos[cgo] = (desc, unidad, precio)
    {11: ('velitas', 'unid', 32, 5), 25: ('azucar', 'gr', 0, 65)} C/ esto vuelve a no
    cgo = int(input('Ingrese código, 0 p/terminar') → a entrar al bucle

```

Desempaquetado de tuplas

```

for cgo in productos:
    print(cgo, *productos[cgo])

```

total = 0

△ cgo = int(input('¿Que lleva? 0 p/salir: '))

while cgo not in productos and cgo != 0: # Si el código no existe, vuelve a pedirlo

```

while cgo != 0:
    Si cgo = 11 | productos[11] = (velitas, unid, 32, 5)
    cant = float(input('Cantidad de %s: ' % (productos[cgo][0])))
    total += cant * productos[cgo][2]
    cgo = int(input('¿Lleva algo más? 0 p/salir: '))

```

Mientras el código ingresado no exista en el dicc productos y no sea cero, sigue pidiéndolo. Vuelve a ejecutar este bloque.

```

while cgo not in productos and cgo != 0:
    cgo = int(input('¿Qué más lleva? 0 p/salir: '))

```

print('Debe abonar: \$%.2f' % total)

Traducción de menú

T dificultad = ('', 'Alta', 'Media', 'Baja')

D dificultad = {1: 'Alta', 2: 'Media', 3: 'Alta'}

platos = []

nomPlato = input('Ingresa un plato, * p/salir:')

while nomPlato != 'x':

opc == 0

while opc == 0:

print('Dificultad')

for i in dificultad:

1-Alta → print('%d - %s' % i, dificultad[i])

opc = int(input('Elegí una opción'))

if opc not in dificultad:

opc = 0

Diccionarios

platos.append([nomPlato, dificultad[opc]]) → guarda

nomPlato = input('Ingresa un plato, * p/salir')

print('Lista de platos')

for p in platos:

print(*p)

for i in range(1, len(dificultad):

print('%d - %s' % (i, dificultad[i]))

opc = int(input())

if opc not in range(1, len(dificultad):

opc = 0

Tuplas

Al usar diccionarios, si ingreso un mismo plato 2 veces, no se duplica. Se pisa el valor

¿Se puede ordenar un diccionario? → Un dict organiza su info en base a la dirección obtenida al aplicar la fn hash a sus claves. No podemos decirle en qué posición irá c/u de ellas

Info ordenada por el nombre de la clave

```
dicci = {}
```

```
print('Datos de clientes, * p/ terminar')
```

```
dni = input('DNI: ')
```

```
while dni != '*':
```

```
    nom = input("Nombre: ")
```

```
    ape = input("Apellido: ")
```

```
    edad = int(input("Edad: "))
```

```
    while edad not in range(18, 130):
```

```
        edad = int(input('Edad entre 18 y 130'))
```

```
    dicci[dni] = [nom, ape, edad]
```

```
    print(dicci) # {'18023569': ['A', 'G', 56], '17895822': ['M', 'S', 66]}
```

```
    dni = input("DNI")
```

`dicciOrden = sorted(dicci)` Aplicando la fn sorted a un diccionario, devuelve una lista

for `pers in dicciOrden`: c/ las CLAVES ordenadas ['17895822', '18023569']

```
print(pers, dicci[pers][0], dicci[pers][1], dicci[pers][2])
```

DNI `dicci['18023569'] = ['A', 'G', 56]`

[0], [1], [2]

❗ Los diccionarios como las listas se comportan c/ las fn de una manera diferente a las otras estructuras de datos.

Cdo envío una lista o diccionario a una fn, NO se copia en el scope de la fn, sino que se le cede a esta última la llave de acceso de la estructura original.

Consecuencias del pasaje de parámetros por referencia

- No necesito devolver una lista o diccionario ya q' las modificaciones se realizan en la estructura original

- Estoy confiando q' modifique mi estructura de forma segura

- Si necesito cargar una lista o {} dentro de una fn, se lo puedo pasar vacío y se irá cargando fuera. Por el contrario, si no lo paso y se crea dentro, es local de la fn y si así debe tener un return p/ poder tener acceso una vez termina la fn

- Si no quiero compartir la estructura org. debo copiarla manualmente

Funciones lambda - Funciones anónimas

-return: Está implícito. Están pensadas y diseñadas p/ ser fn pequeñas. Que sólo tienen una expresión. Fn flecha de js

Fn de orden superior: Pueden aceptar fns como argumentos, devolver fns como rtados y almacenar fn en variables.

Sin lambda

```
def retornar_nota(estudiante):  
    return estudiante[1]
```

```
lista_estudiantes = [  
    ('Ed', 4.2),  
    ('Pe', 2.5)  
]
```

```
lista_ordenada = sorted(lista_estudiantes, key=retornar_nota, reverse=True)
```

Con lambda. Es cómo si se ejecutara en base a la lista →

```
lista_ordenada = sorted(lista_estudiante, key=lambda (arg-q-accepto): arg-q-accepto,  
                        , reverse=True)
```

es lista_estudiantes[i]

Por defecto ordena ascendente

puedo recorrer por pisando el dict